بسم الله الرحمن الرحيم

King Abdulaziz University
Faculty of Computing and information Technology
Computer Science Department

CPCS-214 Computer Architecture and Organization

38 - Pipelining

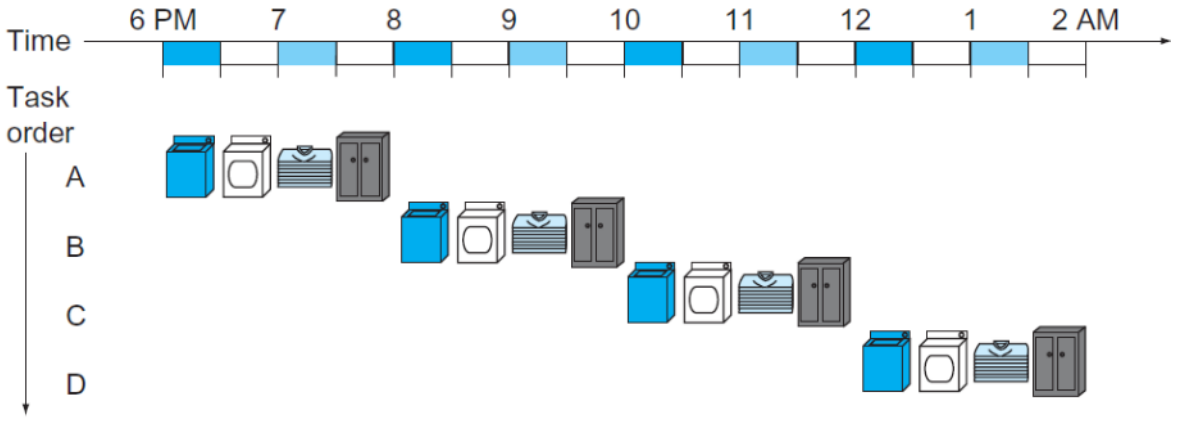
Introduction

- Pipelining is an implementation technique in which multiple instructions are overlapped in execution
- Today, it is nearly universal

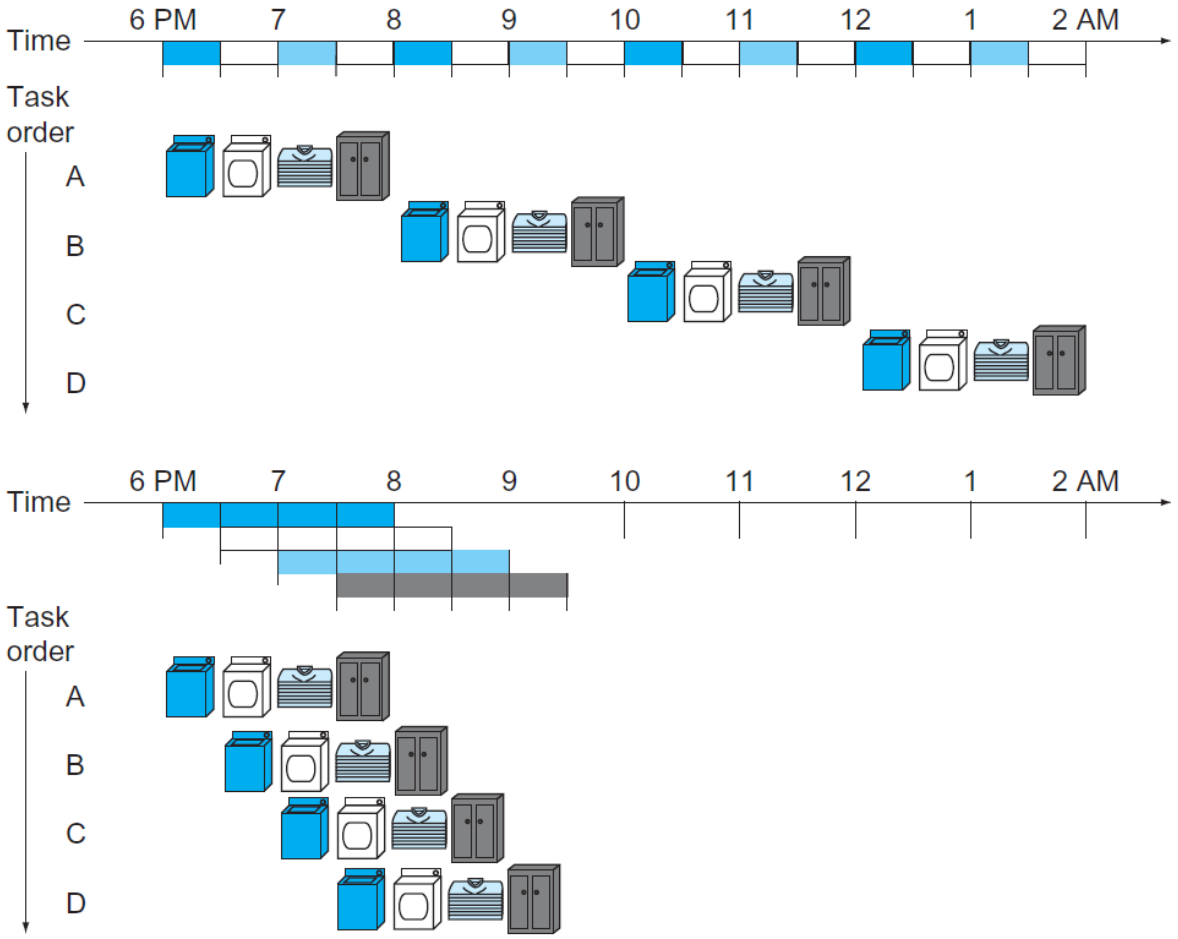
Analogy

- Laundry
- *Nonpipelined* approach
- *Pipelined* approach
  - Takes much less time

Nonpipelined Approach



Pipelined Approach



Pipelined Approach

- All steps, called *stages*, are operating concurrently
- As long as we have separate resources for each stage, we can pipeline tasks
- Reason it is faster: everything is working in parallel
  - so more loads are finished per hour; Improves throughput
- Would not decrease time to complete one load, but when having many loads, improvement in throughput decreases total time to complete the work

Speedup

- If all stages take about the same amount of time and there is enough work to do, then speedup due to pipelining is equal to number of stages in pipeline, in this case 4
  - Pipelined is potentially 4 times faster than nonpipelined
  - 20 loads would take about 5 times as long as 1 load
  - while 20 loads of sequential laundry takes 20 times as long as 1 load

Speedup

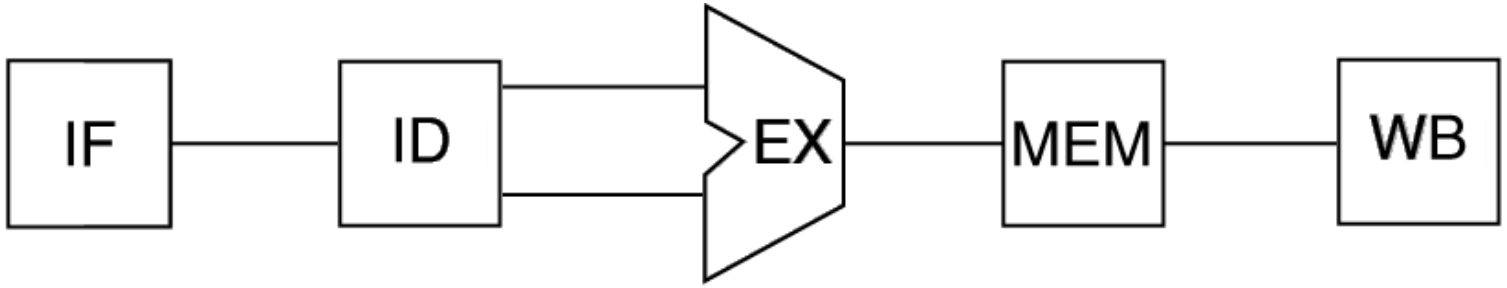
- It’s only 2.3 times faster, because we only show 4 loads
  - At beginning and end of workload, pipeline is not completely full
- This affects performance when the number of tasks is not large compared to the number of stages in pipeline
- If the number of loads is much larger than 4, then the stages will be full most of the time and the increase in throughput will be very close to 4

Processors Pipelining

- Same principles apply to processors
  - Pipeline instruction-execution
- MIPS instructions classically take five steps:
  1. Fetch instruction from memory
  2. Read registers while decoding instruction simultaneously
  3. Execute operation or calculate address
  4. Access operand in data memory
  5. Write result into register

Pipeline Stages

- Datapath is divided into five stages; one step per stage
  - Instruction Fetch
  - Instruction Decode
  - EXexute
  - MEMory
  - Write Back



Stages Graphical Representation

Single-Cycle versus Pipelined Performance

- Limit attention to eight instructions
  - lw, sw, add , sub, and, or, slt, and beq
- All pipeline stages take single clock cycle
  - So clock cycle must be long enough to accommodate the slowest operation

Example

- Compare average time between instructions of a single-cycle implementation, in which all instructions take one clock cycle, to a pipelined implementation. Operation times for major functional units are: 200 ps for memory access, 200 ps for ALU operation, and 100 ps for register file read or write.

Example (cont.)

- Every instruction takes exactly 1 cc, so cc must accommodate the slowest instruction
  - Table shows time required for each instruction

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (`lw`)						
Store word (`sw`)						
R-format (`add`, `sub`, `AND`, `OR`, `slt`)						
Branch (`beq`)						

Example (cont.)

- Every instruction takes exactly 1 cc, so cc must accommodate the slowest instruction
  - Table shows time required for each instruction

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (`lw`)	200 ps	100 ps	200 ps	200 ps	100 ps	
Store word (`sw`)						
R-format (`add`, `sub`, `AND`, `OR`, `slt`)						
Branch (`beq`)						

Example (cont.)

- Every instruction takes exactly 1 cc, so cc must accommodate the slowest instruction
  - Table shows time required for each instruction

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (`lw`)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (`sw`)						
R-format (`add`, `sub`, `AND`, `OR`, `slt`)						
Branch (`beq`)						

Example (cont.)

- Every instruction takes exactly 1 cc, so cc must accommodate the slowest instruction
  - Table shows time required for each instruction

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (`lw`)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (`sw`)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (`add`, `sub`, `AND`, `OR`, `slt`)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (`beq`)	200 ps	100 ps	200 ps			500 ps

Example (cont.)

- Slowest instruction

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (`lw`)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (`sw`)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (`add`, `sub`, `AND`, `OR`, `slt`)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (`beq`)	200 ps	100 ps	200 ps			500 ps

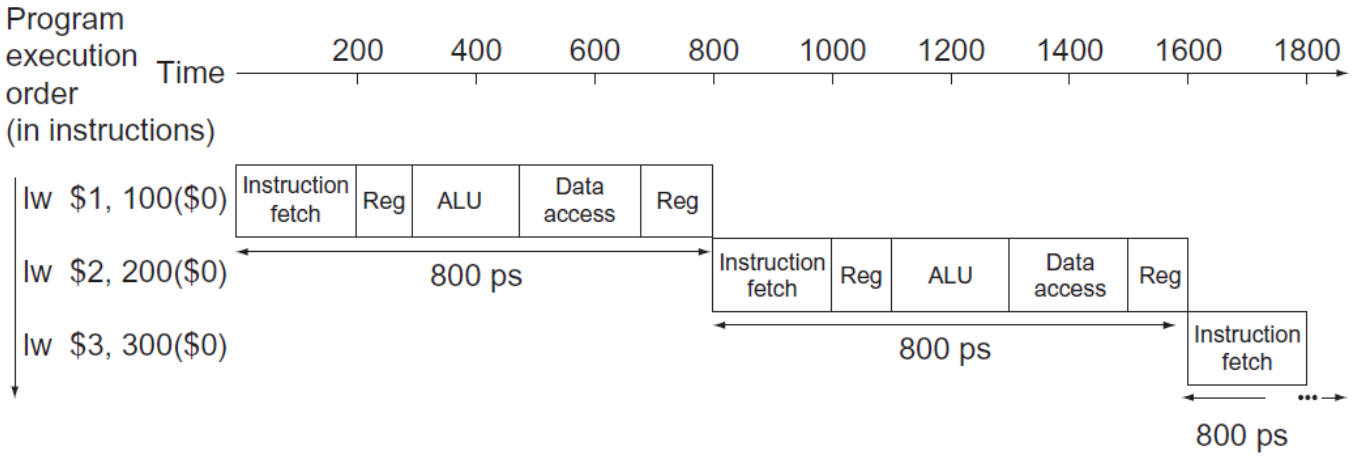
Example (cont.)

- Slowest instruction
  - *lw*
  - So time required for every instruction is 800 ps

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (`lw`)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (`sw`)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (`add`, `sub`, `AND`, `OR`, `slt`)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (`beq`)	200 ps	100 ps	200 ps			500 ps

Example (cont.)

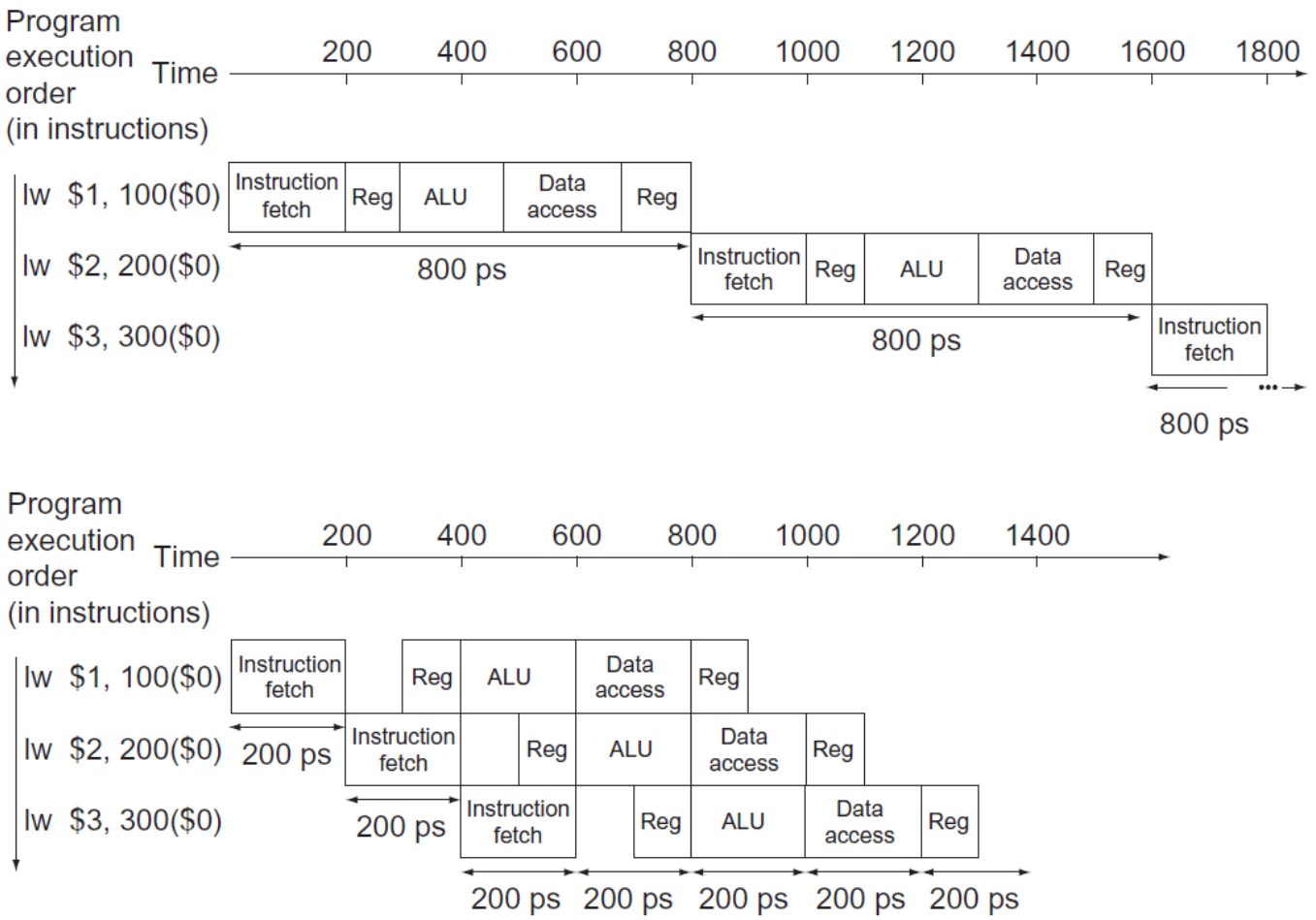
- Time between first and fourth instructions in nonpipelined design is 3 × 800 ps or 2400 ps



Example (cont.)

- Just as single-cycle design must take the worst-case cc of 800 ps, even though some instructions can be as fast as 500 ps,
- Pipelined execution cc must have the worst-case cc of 200 ps, even though some stages take only 100 ps
- Pipelining still offers fourfold performance improvement
  - time between first and fourth instructions is 3 × 200 ps or 600 ps

Example (cont.)



Speedup

- If stages are perfectly balanced, then on pipelined processor

$$\text{Time between instructions}_{\text{pipelined}} = \frac{\text{Time between instruction}_{\text{nonpipelined}}}{\text{Number of pipe stages}}$$

Speedup

- Under ideal conditions and with a large number of instructions, speedup from pipelining is approximately equal to number of pipe stages
  - A 5-stage pipeline is nearly 5 times faster
  - Formula suggests that 5-stage pipeline should offer nearly fivefold improvement over 800 ps nonpipelined time, or 160 ps clock cycle

Speedup

- However, stages may be imperfectly balanced
  - Thus, time per instruction in pipelined processor will exceed the minimum possible, and speedup will be < number of pipeline stages
- Claim of fourfold improvement is not reflected in the total execution time for in the example: 1400 ps versus 2400 ps
  - Because number of instructions is not large
- What would happen if the number of instructions is increased?
  - Extend program to 1,000,003 instructions

Extend Instructions

- In pipelined
  - Each adds 200 ps to execution time
  - Total execution time = 1,000,000 × 200 + 1400 = 200,001,400 ps
- In nonpipelined
  - Each takes 800 ps
  - Total execution time = 1,000,000 × 800 + 2400 = 800,002,400 ps

$$\frac{800,002,400\,ps}{200,001,400\,ps} \simeq \frac{800ps}{200ps} \simeq 4.00$$

Extend Instructions

- Ratio of total execution times for real programs on nonpipelined to pipelined processors is close to ratio of times between instructions

Instruction Throughput

- Pipelining improves performance by *increasing instruction throughput, as opposed to decreasing the execution time of an individual instruction*
- But instruction throughput is the important metric
  - Because real programs execute billions of instructions

Designing Instruction Sets for Pipelining

- First, all instructions are the same length
  - This restriction makes it much easier to
    - Fetch instructions in the first pipeline stage and
    - Decode instructions in the second stage

Designing Instruction Sets for Pipelining

- Second, only we have a few instruction formats, with source register fields being located in same place in each instruction
  - Means that the second stage can begin reading register file at same time that HW is determining what type of instruction was fetched
- If instruction formats were not symmetric, we would need to split stage 2
  - Resulting in 6 pipeline stages

Designing Instruction Sets for Pipelining

- Third, memory operands only appear in loads or stores
  - This restriction means we can use execute stage to calculate memory address and then access memory in the following stage
- If we could operate on operands in memory, stages 3 and 4 would expand to an address stage, memory stage, and then execute stage

Designing Instruction Sets for Pipelining

- Fourth, operands must be aligned in memory
  - Hence, we need not worry about a single data transfer instruction requiring two data memory accesses
  - Requested data can be transferred between processor and memory in a single pipeline stage